NESNE YÖNELİMLİ PROGRAMLAMA-1

Kapsülleme

Erişim belirleyiciler, getter/setter ve @property

ad _ad __ad @property

```
hesap.py
h.bakiye = -4000
AttributeError: property
'bakiye' has no setter

# Bakiye artık korunuyor.
```

Erhan AKAGÜNDÜZ

BU HAFTA

Dersin sonunda neler yapabileceksiniz?

Kapsülleme

Veriyi korumanın neden gerekli olduğunu açıklamak

Erişim seviyeleri

ad, _ad ve __ad arasındaki farkı ve isim karıştırmayı bilmek

Getter/Setter

Okuma ve yazmayı kurallı metotlarla kontrol etmek

@property

Python'un önerdiği zarif yolla doğrulama ve salt okunur nitelik yazmak

Renk kodu: yeşil = public, turuncu = _protected, mor = __private, mavi = @property.

GEÇEN HAFTADAN BUGÜNE

Kapıda kontrol var, ama pencere açık

sorun.py

```
class Urun:
    def __init__(self, ad, fiyat):
        if fiyat <= 0:
            raise ValueError("Fiyat pozitif olmalı")
        self.ad = ad
        self.fiyat = fiyat
u = Urun("Kalem", 15)
u.fiyat = -100          # kimse durdurmadı!
print(u.fiyat)
```

▷ ÇIKTI `-100`

✓ **Kapı (__init__):** negatif fiyatla ürün **doğamaz**.

⚠ **Pencere (u.fiyat = ...):** doğduktan sonra herkes istediğini yazabilir.

Bugünün sorusu

Nesne yaşadığı **her an** geçerli kalsın diye niteliğe erişimi nasıl kontrol ederiz?

KAVRAM

Kapsülleme: içini gizle, kapısını kontrol et

Kapsülleme, bir nesnenin verisini ve o veri üzerindeki işlemleri bir arada tutmak, iç ayrıntıları gizleyip dışarıya yalnızca **kontrollü bir arayüz** sunmaktır.

ATM

Kasaya elinizi sokamazsınız. Sadece ekrandaki düğmeler var; her düğme **kuralları uygular**.

Araba

Motor kaputun altında. Siz direksiyon ve pedalı kullanırsınız; yakıt karışımını **değiştiremezsiniz**.

İlaç kapsülü

İçerik dış etkilerden korunur. Kullanıcı içeriği bilmeden **doğru şekilde** kullanır.

BÖLÜM 1

01

Erişim belirleyiciler

Bir nitelik kime açık? Python'da cevap ismin başındaki alt çizgilerde

public _protected __private isim karıştırma private metotlar

Üç seviye, üç yazım

`self.ad`

Public (açık)

Herkes okuyabilir ve yazabilir. Varsayılan durum.

Mağazanın vitrini

`self._ad`

Protected (korumalı)

Teknik engel yok. Sadece bir **uyarı işareti**: "içe ait, dokunma."

"Personel girebilir" levhası

`self.__ad`

Private (özel)

Python ismi değiştirir; dışarıdan **aynı adla** erişilemez.

Kilitli kasa odası

Java ve C#'ta **public / protected / private** anahtar kelimeleri vardır. Python'da yoktur; bunun yerine **isimlendirme geleneği** kullanılır.

SEVİYE 1 · PUBLIC

Alt çizgisiz isim: herkese açık

public.py

```python
class Ogrenci:
    def __init__(self, ad):
        self.ad = ad          # public

o = Ogrenci("Ada")
print(o.ad)
o.ad = "Furkan"
print(o.ad)
```

▷ ÇIKTI

```
Ada
Furkan
```

Ne zaman public?

Değeri ne olursa olsun nesnenin bozulmayacağı veriler: **ad, açıklama, renk** gibi serbest alanlar.

Şimdiye kadar yazdığımız **her nitelik** public'ti.

⚠ Bir kurala bağlı veriler (fiyat > 0, not 0–100) public bırakılırsa kural **atlanabilir**.

SEVİYE 2 · _PROTECTED

Tek alt çizgi: kilit değil, uyarı levhası

protected.py

```
class Hesap:
    def __init__(self, sahip, bakiye):
        self.sahip = sahip
        self._bakiye = bakiye
h = Hesap("Mizgin", 1000)
print(h._bakiye)     # çalışır, ama ayıp!
h._bakiye = 5
print(h._bakiye)
```

▷ ÇIKTI

```
1000
5
```

Programcılar arası anlaşma

Python hiçbir şeyi engellemez. **_bakiye** şunu söyler: “Bu iç ayrıntıdır; sınıfın dışından kullanma, ileride değişebilir.”

Sınıfın kendi metotları ve **alt sınıflar** (9. hafta) kullanabilir.

Editörler ve kod analiz araçları dışarıdan kullanımı **uyarıyla** işaretler.

Python topluluğunda en yaygın **“gizli”** işaretidir.

SEVİYE 3 · __PRIVATE

Çift alt çizgi: dışarıya kapalı

private.py

```
class Hesap:
    def __init__(self, bakiye):
        self.__bakiye = bakiye

    def goster(self):
        return self.__bakiye  # içeriden: serbest
h = Hesap(1000)
print(h.goster())
try:
    print(h.__bakiye)
except AttributeError as e:
    print("Hata:", e)
```

▷ ÇIKTI

```
1000
Hata: 'Hesap' object has no attribute '__bakiye'
```

Sınıfın içinden: self.__bakiye çalışır

Dışarıdan: h.__bakiye hata verir

Ama dikkat: sonunda da iki alt çizgi olan **__init__** gibi isimler private **değildir**; onlar özel (dunder) metotlardır.

PERDE ARKASI · İSİM KARIŞTIRMA

Python gizlemez, ismi değiştirir

mangling.py

```python
class Hesap:
    def __init__(self, bakiye):
        self.__bakiye = bakiye

h = Hesap(1000)
print(h.__dict__)
print(h._Hesap__bakiye)  # arka kapı
```

▷ ÇIKTI

```
{'_Hesap__bakiye': 1000}
1000
```

SINIFIN İÇİNDE YAZILAN

`self.__bakiye`



PYTHON'UN SAKLADIĞI

`self._Hesap__bakiye`

Kural: __isim → _SinifAdi__isim. Asıl amaç gizlilik değil, alt sınıflarda **isim çakışmasını** önlemek.

TUZAK · DIŞARIDAN __ATAMA

Bakiyeyi sıfırladım sandım, ama...

tuzak.py

```
class Hesap:
    def __init__(self, bakiye):
        self.__bakiye = bakiye
    def goster(self):
        return self.__bakiye
h = Hesap(1000)
h.__bakiye = 0      # yeni nitelik oluşur!
print(h.goster())
print(h.__dict__)
```

▷ ÇIKTI

```
1000
{'_Hesap__bakiye': 1000, '__bakiye': 0}
```

Ne oldu?

Sınıfın dışındaki kod ismi karıştırılmaz. **h.__bakiye = 0** yepyeni, ilgisiz bir nitelik yarattı. Gerçek bakiye (**_Hesap__bakiye**) hâlâ 1000.

İyi haber: gerçek veri korundu. Kötü haber: hata vermedi, **sessizce** yanlış bir şey oldu.

PYTHON FELSEFESİ

Kilit değil, sözleşme

> **“We are all consenting adults here.”**
>
> “Burada hepimiz ne yaptığını bilen yetişkinleriz.” · Python topluluğunda yaygın bir söz

Java, C#

private’a dışarıdan erişim **derleme hatası**. Derleyici kapıyı kilitler.

Python

Alt çizgiler bir **niyet beyanı**. Kapıyı açabilirsin; ama açarsan sonuç senin sorumluluğunda.

Asıl koruma isimlerden değil, birazdan göreceğimiz **getter/setter ve @property** ile kurulan kurallardan gelir.

PRIVATE METOTLAR

Metotlar da gizlenebilir

sepet.py

```
class Sepet:
    def __init__(self):
        self.__urunler = []
    def ekle(self, ad, fiyat):
        self.__urunler.append((ad, fiyat))
    def toplam(self):
        t = sum(f for _, f in self.__urunler)
        return self.__indirim_uygula(t)
    def __indirim_uygula(self, tutar):
        return tutar * 0.9 if tutar > 100 else tutar

s = Sepet()
s.ekle("Kitap", 80); s.ekle("Defter", 40)
print(s.toplam())
try: s.__indirim_uygula(200)
except AttributeError as e: print("Hata:", e)
```

▷ ÇIKTI

```
108.0
Hata: 'Sepet' object has no attribute
'__indirim_uygula'
```

SEPETİN ARAYÜZÜ

`ekle()` `toplam()`

İÇ AYRINTI

`__urunler` `__indirim_uygula()`

İndirim kuralı yarın değişse bile **arayüz aynı** kalır; sepeti kullanan kod etkilenmez.

BÖLÜM 2

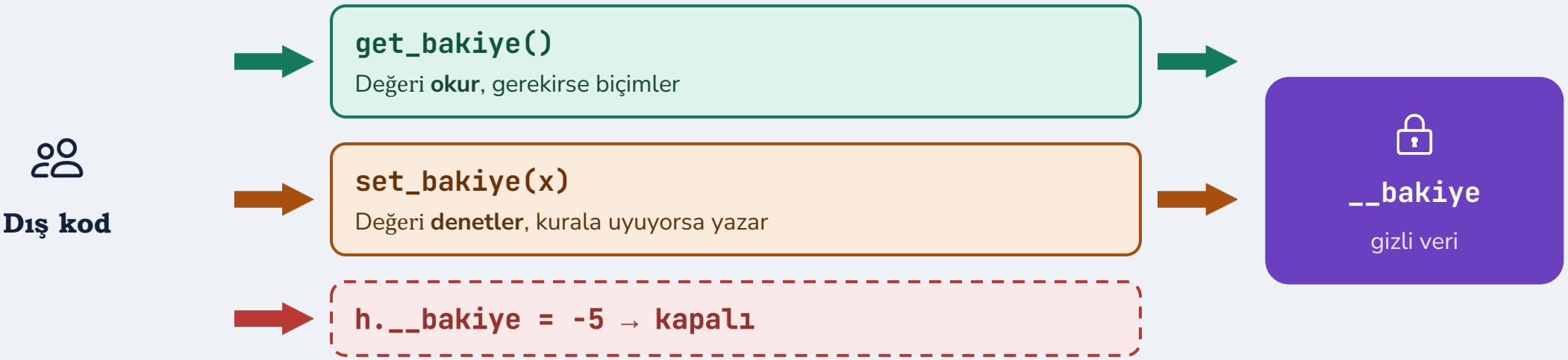
02

Getter ve setter

Veriyi gizledik; şimdi ona kurallı bir kapı açıyoruz

- get_x() okur
- set_x() denetler ve yazar
- salt okunur nitelik
- Java tarzının sorunu

Kasaya doğrudan değil, gişeden ulaş



Getter (erişici) okur, **setter** (değiştirici) yazar. Kural tek bir yerde durur: setter'ın içinde.

GETTER/SETTER · KOD

Kural setter’da, bakiye hep geçerli

```
class Hesap:
    def __init__(self, sahip, bakiye):
        self.sahip = sahip
        self.__bakiye = 0
        self.set_bakiye(bakiye)
    def get_bakiye(self):
        return self.__bakiye
    def set_bakiye(self, yeni):
        if yeni < 0:
            raise ValueError("Bakiye negatif olamaz")
        self.__bakiye = yeni
h = Hesap("Aycan", 1000)
print(h.get_bakiye())
h.set_bakiye(250); print(h.get_bakiye())
try: h.set_bakiye(-50)
except ValueError as e: print("Hata:", e)
print(h.get_bakiye())
```

▷ ÇIKTI

```
1000
250
Hata: Bakiye negatif olamaz
250
```

Dikkat edilecek iki nokta

1. Hatalı değer reddedildi; bakiye **250’de kaldı**.

2. __init__ de **set_bakiye()** kullanıyor; kural tek yerde, iki kez yazılmadı.

Setter yazmazsan, değer değiştirilemez

salt_okunur.py

```python
class Ogrenci:
    def __init__(self, ad, numara):
        self.ad = ad
        self.__numara = numara

    def get_numara(self):   # sadece getter
        return self.__numara
o = Ogrenci("Ada", 101)
print(o.get_numara())
try: o.set_numara(999)
except AttributeError as e: print("Hata:", e)
```

▷ ÇIKTI

```
101
Hata: 'Ogrenci' object has no attribute
'set_numara'
```

Neden salt okunur?

Öğrenci numarası bir kez verilir ve **hiç değişmemelidir**. Getter var, setter yok: dışarısı okuyabilir, ama yazamaz.

Diğer örnekler: **T.C. kimlik no**, **hesap açılış tarihi**, **sipariş numarası**.

Çalışıyor, ama okuması yorucu

Doğrudan nitelik

```
h.bakiye = h.bakiye + 100
```

Okunaklı ama korumasız.

Getter / setter

```
h.set_bakiye(h.get_bakiye() + 100)
```

Korumalı ama hantal.

Bir de şu sorun var

Bugün **h.bakiye** kullanan 200 satır kodunuz var. Yarın kural eklemek için hepsini **get_/set_** çağrısına çevirmeniz gerekir.

İkisinin en iyisi?

Yazımı **h.bakiye** gibi, davranışı **setter** gibi olan bir şey. Python'daki adı: **@property**.

BÖLÜM 3

03

@property

Nitelik gibi görünen, metot gibi çalışan yapı

getter ve .setter

__init__ ile birlikte

salt okunur hesaplanan değer

dönüşüm

liste sızıntısı

Aynı isimde iki metot, iki etiket

```
class Urun:

    @property
    def fiyat(self):
        return self._fiyat

    @fiyat.setter
    def fiyat(self, yeni):
        # kontrol et…
        self._fiyat = yeni
```

`@property`

Altındaki metot **getter** olur. Adı (fiyat) dışarıdan nitelik gibi kullanılır.

`@fiyat.setter`

Aynı isimli ikinci metot **setter** olur. Dekoratördeki ad getter ile aynı olmalı.

`self._fiyat`

Verinin asıl saklandığı yer. Property ile **farklı isimde** olmalı.

@PROPERTY · KOD

Pencere de artık kontrol altında

```
class Urun:
    def __init__(self, ad, fiyat):
        self.ad = ad
        self.fiyat = fiyat    # setter'dan geçer!
    @property
    def fiyat(self):
        return self._fiyat
    @fiyat.setter
    def fiyat(self, yeni):
        if yeni <= 0:
            raise ValueError("Fiyat pozitif olmalı")
        self._fiyat = yeni
u = Urun("Kalem", 15); print(u.fiyat)
u.fiyat = 20; print(u.fiyat)
try: u.fiyat = -100
except ValueError as e: print("Hata:", e)
try: Urun("Hatalı", 0)
except ValueError as e: print("Hata:", e)
```

▷ ÇIKTI

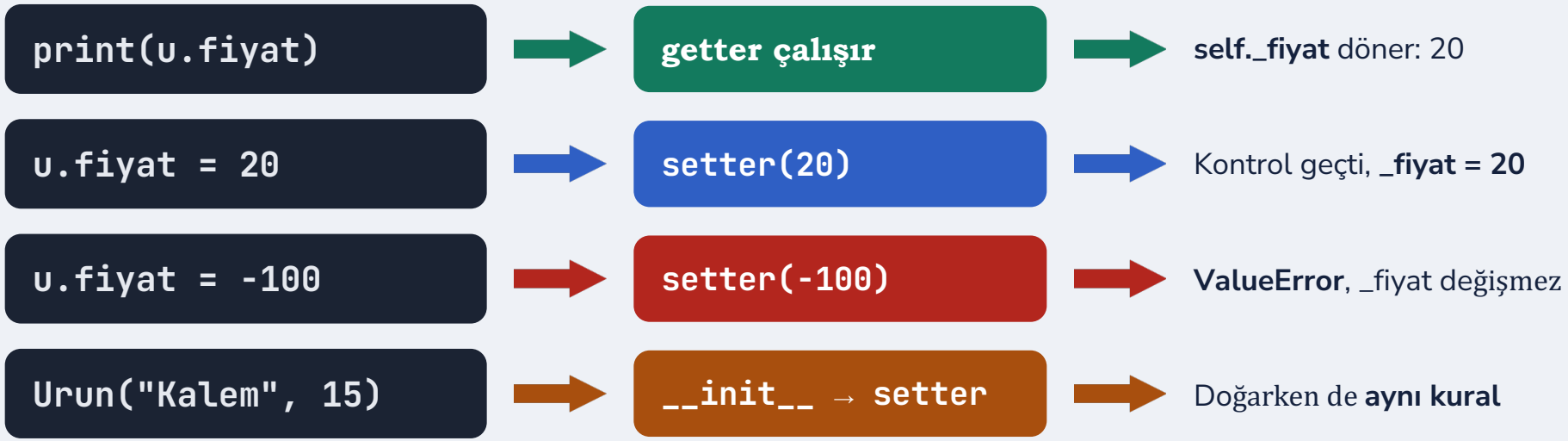
```
15
20
Hata: Fiyat pozitif olmalı
Hata: Fiyat pozitif olmalı
```

Kullanım hiç değişmedi

Dışarıdaki kod hâlâ **u.fiyat** ve **u.fiyat = 20** yazıyor. Ama her atama artık setter'dan geçiyor.

✓ 3. slayttaki **u.fiyat = -100** açığı kapandı.

Bir nokta yazdın, Python bir metot çağırdı



Kullanan kod bir nitelik görüyor; sınıf ise her erişimi bir metotla **yönetiyor**.

TUZAK · SONSUZ DÖNGÜ

Property ve veri aynı adı taşırsa

sonsuz.py

```
class Urun:
    def __init__(self, fiyat):
        self.fiyat = fiyat
    @property
    def fiyat(self):
        return self.fiyat     # _ yok!
    @fiyat.setter
    def fiyat(self, yeni):
        self.fiyat = yeni     # _ yok!
Urun(15)
```

▷ ÇIKTI

```
RecursionError: maximum recursion depth exceeded
```

Kendi kendini çağıran setter

`self.fiyat = yeni`

⬇ setter'ı tekrar çağırır

`self.fiyat = yeni`

⬇ ... yaklaşık 1000 kez, sonra çöker

Kural

Property içinde veriyi **self._fiyat** ile oku ve yaz. **__init__** ise **self.fiyat** yazsın.

Setter'sız property: salt okunur

daire.py

```python
class Daire:
    def __init__(self, yaricap):
        self.yaricap = yaricap

    @property
    def alan(self):
        return round(3.14159 * self.yaricap ** 2, 2)

d = Daire(2)
print(d.alan)
d.yaricap = 3
print(d.alan)
try: d.alan = 100
except AttributeError as e: print("Hata:", e)
```

▷ ÇIKTI

```
12.57
28.27
Hata: property 'alan' of 'Daire' object has no
setter
```

3. haftadaki sorun çözüldü

Dikdörtgen örneğinde alan **eskiyordu**. Burada yarıçap değişince alan da kendiliğinden güncellendi; çünkü her okumada yeniden hesaplanıyor.

Yazım **d.alan**: parantez yok, nitelik gibi.

@PROPERTY · DÖNÜŞÜM

Tek veri, iki görünüm

sicaklik.py

```
class Sicaklik:
    def __init__(self, celsius):
        self.celsius = celsius
    @property
    def fahrenheit(self):
        return self.celsius * 9 / 5 + 32
    @fahrenheit.setter
    def fahrenheit(self, f):
        self.celsius = (f - 32) * 5 / 9
t = Sicaklik(25)
print(t.fahrenheit)
t.fahrenheit = 212
print(t.celsius)
```

▷ ÇIKTI

```
77.0
100.0
```

25 °C → **77 °F**

saklanan — hesaplanan

Yalnızca **celsius** saklanıyor. Fahrenheit hem okunabiliyor hem yazılabiliyor; ama iki ayrı veri olmadığı için **çelişki imkânsız**.

Setter sadece reddetmez, düzeltebilir de

profil.py

```python
class Profil:
    def __init__(self, eposta):
        self._eposta = eposta
    @property
    def eposta(self):
        return self._eposta
    @eposta.setter
    def eposta(self, yeni):
        if "@" not in yeni:
            raise ValueError("Geçersiz e-posta")
        self._eposta = yeni.lower()
p = Profil("ada@okul.edu")
p.eposta = "ADA@Yeni.com"
print(p.eposta)
try: p.eposta = "adaokul"
except ValueError as e: print("Hata:", e)
```

▷ ÇIKTI

```
ada@yeni.com
Hata: Geçersiz e-posta
```

Reddet: “@” yoksa hata ver.

Düzenle: büyük harfleri küçült, tek biçimde sakla.

⚠ Bu örnekte __init__ **self._eposta** yazıyor; yani doğumda kontrol yok. Düzeltmesi ödevde.

TUZAK · LİSTE SIZINTISI

Kapıyı kilitledik, anahtarı da verdik

sizinti.py

```
class Ogrenci:
    def __init__(self, ad):
        self.ad = ad
        self.__notlar = []
    def not_ekle(self, n):
        if not 0 <= n <= 100:
            raise ValueError("Not 0-100 arası olmalı")
        self.__notlar.append(n)
    @property
    def notlar(self):
        return self.__notlar    # dikkat!
o = Ogrenci("Ada")
o.not_ekle(80)
o.notlar.append(500)   # kuralı atladı!
print(o.notlar)
```

▷ ÇIKTI `[80, 500]`

Ne oldu?

Getter, gizli listenin **kendisini** döndürdü (1. haftadaki referans konusu). Dışarıdaki kod o listeyi doğrudan değiştirdi; **not_ekle()**'deki kural hiç çalışmadı.

`o.notlar` → `__notlar` aynı liste

ÇÖZÜM · SALT OKUNUR KOPYA

Orijinali değil, demet kopyasını ver

kopya.py

```
# ... sınıfın geri kalanı aynı
    @property
    def notlar(self):
        return tuple(self.__notlar)
o = Ogrenci("Ada")
o.not_ekle(80)
try:
    o.notlar.append(500)
except AttributeError as e:
    print("Hata:", e)
print(o.notlar)
```

▷ ÇIKTI

```
Hata: 'tuple' object has no attribute 'append'
(80,)
```

Neden işe yaradı?

tuple() yeni bir demet oluşturur. Demet değiştirilemez (1. hafta); **append** metodu bile yok. Orijinal liste dokunulmadan kalır.

Getter döndürür	Sonuç
self.__notlar	Sızıntı: dışarısı değiştirir
self.__notlar.copy()	Güvenli kopya (liste)
tuple(self.__notlar)	Güvenli ve salt okunur

Değişiklik yalnızca **not_ekle()** kapısından girebilir.

KARŞILAŞTIRMA

Üç yol, aynı bakiye

	Public nitelik	get_ / set_ metotları	@property
Okuma	h.bakiye	h.get_bakiye()	h.bakiye
Yazma	h.bakiye = 50	h.set_bakiye(50)	h.bakiye = 50
Doğrulama	Yok	Var	Var
Salt okunur	Yapılamaz	Setter yazma	Setter yazma
Okunaklılık	Yüksek	Düşük	Yüksek
Sonradan eklemek	–	Tüm kodu değiştirir	Kod aynı kalır

☆ **Python yolu:** önce public başla; kural gerektiği gün @property'ye geç. Kullanan kod fark etmez bile.

KARAR VER

Bu nitelik nasıl yazılmalı?

Ürün açıklaması

Her metin geçerli

```
self.aciklama
```

Ürün fiyatı

Pozitif olmalı, değişebilir

```
@property + setter
```

Öğrenci numarası

Bir kez verilir, değişmez

```
@property (setter yok)
```

Not ortalaması

Notlardan hesaplanır

```
@property (hesaplanan)
```

Önbellek, sayaç

Sadece sınıf içi kullanım

```
self._onbellek
```

Hesap bakiyesi

Sadece metotlarla değişmeli

```
__bakiye + @property
```

Soru: **“Bu değer yanlış olursa nesne bozulur mu?”** Hayır → public. Evet → kontrol et.

UYGULAMA · MİNİ PROJE

Tam kapsüllenmiş banka hesabı

```
class BankaHesabi:
    __faiz_orani = 0.02
    def __init__(self, sahip, bakiye=0):
        self.sahip = sahip
        self.__bakiye = 0; self.__gecmis = []
        if bakiye:
            self.yatir(bakiye)
    @property
    def bakiye(self): return self.__bakiye
    @property
    def gecmis(self): return tuple(self.__gecmis)
```

sahip: serbest

bakiye, gecmis: salt okunur

__...: iç ayrıntı

```
    def yatir(self, miktar):
        self.__kontrol(miktar)
        self.__bakiye += miktar
        self.__gecmis.append(f"+{miktar}")
    def cek(self, miktar):
        self.__kontrol(miktar)
        if miktar > self.__bakiye:
            raise ValueError("Yetersiz bakiye")
        self.__bakiye -= miktar
        self.__gecmis.append(f"-{miktar}")
    def faiz_ekle(self):
        faiz = self.__bakiye * self.__faiz_orani
        self.yatir(round(faiz, 2))
    @staticmethod
    def __kontrol(miktar):
        if miktar <= 0:
            raise ValueError("Miktar pozitif olmalı")
```

Hesabı bozmaya çalışalım

```
h = BankaHesabi("Ömer", 1000)
h.cek(300)
h.faiz_ekle()
print(h.bakiye)
print(h.gecmis)

# saldırılar
try: h.cek(5000)
except ValueError as e: print("Hata:", e)
try: h.yatir(-5)
except ValueError as e: print("Hata:", e)
try: h.bakiye = 1_000_000
except AttributeError as e: print("Hata:", e)
```

▷ ÇIKTI

```
714.0
('+1000', '-300', '+14.0')
Hata: Yetersiz bakiye
Hata: Miktar pozitif olmalı
Hata: property 'bakiye' of 'BankaHesabi' object has
no setter
```

Deneme	Kim durdurdu?
cek(5000)	cek() içindeki kural
yatir(-5)	__kontrol()
bakiye = 1_000_000	setter'ı olmayan property

SÖZLÜK

Bu haftanın terimleri

Terim	İngilizce	Anlamı
Kapsülleme	encapsulation	Veriyi gizleyip erişimi kontrollü bir arayüzle sağlamak
Erişim belirleyici	access modifier	Bir üyenin kimlere açık olduğunu belirten işaret
Korumalı üye	protected	_ad: “içe ait” gelenek işareti
Özel üye	private	__ad: ismi karıştırılan, dışarıya kapalı üye
İsim karıştırma	name mangling	__ad → _Sinif__ad dönüşümü
Erişici / değiştirici	getter / setter	Değeri okuyan / denetleyip yazan metot
Özellik	property	Nitelik gibi kullanılan, metot gibi çalışan yapı
Salt okunur	read-only	Okunabilen ama dışarıdan yazılamayan değer

KONTROL LİSTESİ

Kapsüllemeyi doğru yaptım mı?

- Kurala bağlı her nitelik **korunuyor** mu?
- __init__ property'yi (**self.fiyat**) kullanıyor mu?
- Değişmemesi gereken değerlerin **setter'ı yok** mu?
- İç yardımcı metotlar **_ veya __** ile işaretli mi?
- Doğrulama **setter'da**, tek bir yerde mi?
- Property içinde veri **_alt_cizgili** isimle mi tutuluyor?
- Getter listeleri **kopya veya tuple** olarak mı veriyor?
- Gereksiz yere **her şeyi** gizlemedim mi?

ÇIKTIYI TAHMİN ET

Ekrana ne yazılır?

SORU 1

```
class A:
    def __init__(self):
        self.__x = 1
        self._y = 2

a = A()
print(hasattr(a, "_y"),
      hasattr(a, "__x"),
      hasattr(a, "_A__x"))
```

True False True: __x, _A__x adıyla saklanır.

SORU 2

```
class Kutu:
    def __init__(self, v): self.deger = v
    @property
    def deger(self): return self._d
    @deger.setter
    def deger(self, v): self._d = max(0, v)

k = Kutu(-7)
k.deger = k.deger + 5
print(k.deger)
```

5: setter, -7'yi 0'a çekti; 0 + 5 = 5.

DİKKAT

Sık yapılan altı hata

`h.__bakiye = 0`

Dışarıda karıştırma olmaz; yeni, ilgisiz bir nitelik açılır. Gerçek bakiye değişmez.

`return self.fiyat`

Property kendini çağırır: **RecursionError**. Veriyi **self._fiyat**'ta tutun.

`@fiyatt.setter`

Setter'ın adı ve dekoratördeki ad, property ile **birebir aynı** olmalı.

`return self.__notlar`

Liste referansı sızar, dışarıdan append edilir. **tuple(...)** veya kopya döndürün.

`self._fiyat = f`

__init__'te doğrudan yazmak doğrulamayı atlar. **self.fiyat = f** yazın.

`u.fiyat()`

Property parantezsiz okunur. Çıktı: **'int' object is not callable**

ÖZET

Bu hafta neler öğrendik?

Erişim seviyeleri

ad, _ad, __ad: açık, içe ait, dışarıya kapalı.

İsim karıştırma

__ad, _Sinif__ad olur. Kilit değil, kazaya karşı perde.

Getter / Setter

Okuma ve yazma tek kapıdan; kural setter'da.

@property

Nitelik sözdizimi, metot gücü. Setter yoksa salt okunur.

Kapsülleme, nesnenin **kendi kurallarını kendisinin korumasıdır**.

ÖDEV

Haftaya kadar

1 • E-postayı onar

Kullanici sınıfında __init__ setter'ı kullansın; geçersiz e-posta doğumda reddedilsin.

2 • Not defteri

Ogrenci: not_ekle (0–100 arası), notlar tuple olarak, ortalama hesaplanan property.

3 • Dikdörtgen

en ve boy pozitif setter'larla; alan ve cevre salt okunur.

4 • Hesaba saldır

BankaHesabi'nı bozmanın 3 yolunu dene; hangisi başardı, neden?

GELECEK HAFTA · 6

Magic metotlar ve operatör aşırı yükleme

```
__str__
__eq__
__add__
```

print(h) neden <...object at 0x...> yazıyor?